[GUIDE] Module 08 -- Personal Contribution, Code Ownership & Technical Gaps

CONFIDENTIAL & PRIVATE STUDY MATERIAL
Target Roles: AI Engineer, Applied AI Engineer, Backend AI Engineer
<- Back to Study Notes Index - Previous Module: Interview Answers <-

[INDEX] Executive Summary & Purpose

This document serves as your **strategic boundary and interview defense guide**. In high-stakes technical interviews, interviewers frequently probe candidate authenticity by asking deep, pointed questions about specific lines of code, architectural decisions, and failure modes.

Being 100% honest, articulate, and technically precise about **what you personally built**, **what you did not build**, and **where the system has real-world limitations** demonstrates high senior engineering maturity. Claiming code you did not write is the fastest way to fail a technical interview; conversely, demonstrating deep ownership of the backend and AI architecture while clearly articulating future scaling paths establishes immense credibility.

1. [SECURITY] Explicit Ownership Boundaries

What You Personally Owned & Developed (Core Focus)

You own the **Backend Architecture**, **Data Engineering**, and **Applied AI Systems**:

Functional Domain	Specific Responsibilities & Artifacts Owned
FastAPI Backend Architecture	API lifecycle, router modularization (`/api/auth`, `/api/resumes`, `/api/jobs`, `/api/candidates`, `/api/recruiter`), Pydantic v2 validation contracts, custom exception handling, dependency injection (`Depends(get_db)`, `Depends(get_current_user)`, `RoleChecker`).
Database & ORM Layer	PostgreSQL 16 schema design across 10 tables, SQLAlchemy 2.0 declarative models, relational integrity constraints (`UniqueConstraint`, foreign keys with cascading actions), connection pooling configuration (`QueuePool`, `pool_pre_ping`, `pool_recycle`), and 7 complete Alembic migration revisions.
Document Processing Engine	Integration of IBM Docling (`DocumentConverter`) for visual layout segmentation, multi-column reading order reconstruction, PDF magic byte verification (`%PDF-`), and canonical skill extraction/normalization algorithms.
Hybrid ATS Scoring System	Mathematical formulation and implementation of the 5-component deterministic scoring algorithm (*`0.45 * S_{req} + 0.15 * S_{pref} + 0.20 * S_{exp} + 0.10 * S_{sem} + 0.10 * S_{comp}`*) ensuring zero hallucination in numerical ratings.
Applied AI & RAG Pipeline	Google Gemini 2.5 Flash integration via `google-genai` SDK, `text-embedding-004` vector generation, overlapping document chunking (500 chars / 100 overlap), in-memory cosine similarity retrieval, constrained JSON generation, and the deterministic Substring Citation Verification gate.
Security & Operational Reliability	Argon2id password hashing, JWT creation/validation with custom claims (`role`, `sub`), sliding-window in-memory rate limiting, CORS configuration, correlation ID propagation (`X-Request-ID`), structured JSON logging with automated PII scrubbing, and dual file storage (Local vs Cloudflare R2).
Containerization & Orchestration	Authoring multi-stage `backend/Dockerfile` with non-root security (`appuser`), configuring the 4-service `docker-compose.yml` (`db`, `migration`, `backend`, `frontend`), and configuring container health checks and volume persistence.

What You Did NOT Build (Strictly Disclaimed)

[IMPORTANT]
Do NOT claim frontend development. If asked about the frontend, clearly state that your focus was backend, database, and AI systems, and that you interfaced with the frontend purely through standardized REST API contracts (OpenAPI/Swagger).

- **Frontend React Application:** You did **not** build the React/Vite single-page application (`frontend/src/`), component styling, state management, or UI design.
- **Frontend Containerization Details:** While you understand the Nginx reverse-proxy setup, state that your ownership was the backend service and multi-container orchestration.
- **Third-Party Open-Source Models:** You did **not** train the foundational LLM (Gemini 2.5 Flash), the embedding model (`text-embedding-004`), or the Docling neural networks. You engineered the application integration, prompt guardrails, retrieval pipelines, and deterministic verification gates.

2. [TARGET] Verified vs. Unverified Features Matrix

To avoid making claims that do not hold up against the codebase, memorize this exact implementation reality:

```
+-------------------------------------------------------------------------+
|                        HIRELY FEATURE STATUS                            |
+-------------------------------+-----------------------------------------+
| [OK] FULLY IMPLEMENTED          | [WARN] PARTIALLY / SIMPLIFIED           |
|  - Argon2id + JWT Auth & RBAC  |  - RAG Multi-Turn Memory (Single-turn)|
|  - PostgreSQL 10 Models        |  - Vector Storage (In-memory Python,  |
|  - 7 Alembic Migrations        |    not pgvector/Pinecone)             |
|  - Docling Layout PDF Parsing  |  - Rate Limiter (In-memory deque,     |
|  - Hybrid 5-Factor ATS Scoring |    not Redis cluster)                 |
|  - Recruiter Candidate Ranking |  - Storage Service (Local disk primary|
|  - Gemini 2.5 Flash LLM        |    with optional R2 config)           |
|  - Substring Citation RAG      |  - Background Tasks (Sync execution,  |
|  - Structured JSON Logging     |    no Celery / RabbitMQ queue)        |
|  - Docker 4-Service Compose    |  - Refresh Tokens (Access token only) |
+-------------------------------+-----------------------------------------+
```

---

3. [NOTE] Honest, High-Impact Interview Answer Templates

When interviewers probe unfamiliar areas, peripheral tools, or system limitations, use these calibrated templates to answer with confidence, honesty, and technical depth:

Template 1: When Asked About Code or Modules You Didn't Personally Write (e.g., Frontend)

> *"In Hirely, my technical domain was strictly focused on Backend Engineering, Database Architecture, and Applied AI Pipelines. The frontend is a React/Vite application that was developed collaboratively. My interaction with the frontend was centered on defining and upholding the OpenAPI REST contracts, standardizing error envelopes, managing CORS headers, and ensuring that our asynchronous backend endpoints delivered sub-second responses for frontend state updates. I'm happy to dive deep into any aspect of the API design, data schemas, or AI integration."*

Template 2: When Asked Why You Didn't Use a Dedicated Vector Database (e.g., pgvector / Pinecone)

> *"That was an intentional engineering trade-off for the MVP phase. For a single candidate's resume, Docling produces approximately 20 to 50 semantic chunks. Computing cosine similarity across 50 768-dimensional normalized vectors in NumPy takes less than 3 milliseconds on standard CPU hardware. Introducing pgvector or a managed service like Pinecone would have added operational overhead and external dependencies without tangible latency benefits at our initial scale. However, as candidate volume scales to thousands of applicants, linear scanning in Python becomes an `O(N)` memory and CPU bottleneck. The clear production evolution is to introduce* **`pgvector`** *with HNSW indexing directly into our PostgreSQL database via an Alembic migration."*

Template 3: When Asked Why Document Parsing is Synchronous Instead of in a Background Queue

> *"Currently, Docling parsing executes within the request lifecycle. For standard resumes, this takes approximately 1.5 to 2.5 seconds. For an MVP, this simplified the architecture by eliminating the need for a distributed task broker like RabbitMQ or Celery. However, this is a known bottleneck: under burst traffic, CPU-bound PDF parsing can exhaust ASGI worker threads and degrade event-loop concurrency. In our production scaling roadmap, the very first architectural upgrade is moving document ingestion to Celery or Temporal workers, where the client receives an immediate* **`202 Accepted`** *with a task ID and receives real-time updates via WebSockets."*

Template 4: When Asked About Something You Genuinely Do Not Know

> *"I haven't implemented that specific pattern in production yet, but based on my understanding of the underlying principles, here is how I would approach it: [Explain mental model, trade-offs, and how you would research/verify the solution]. For instance, in Hirely, whenever we encountered architectural unknowns -- such as preserving reading order in multi-column PDFs -- we evaluated the trade-offs empirically before committing to the solution."*

---

4. [WARN] Known Technical Debt & Production Gaps

Be prepared to openly discuss these 6 architectural trade-offs. Interviewers love asking: *"What is the weakest part of your architecture, and how would you fix it?"*

Gap 1: In-Memory Sliding-Window Rate Limiting

- **Current State:** Implemented via Python **`collections.deque`** mapped by client IP in process memory.
- **Limitation:** In a multi-worker Uvicorn deployment or horizontal Kubernetes cluster, memory is isolated per process. A client routed round-robin across 4 workers gets *4x* the intended rate limit.
- **Production Fix:** Migrate to a centralized Redis cluster utilizing atomic Lua scripts (**`ZADD`**, **`ZREMRANGEBYSCORE`**, **`ZCARD`**) to enforce cluster-wide rate limits.

Gap 2: Synchronous PDF Ingestion

- **Current State:** File upload, validation, Docling layout analysis, and skill extraction execute synchronously inside the **`POST /api/resumes/upload`** HTTP handler.

- **Limitation:** saturates CPU cores; concurrent uploads risk ASGI thread pool starvation and reverse-proxy 504 Gateway Timeouts.
- **Production Fix:** Decouple via Celery/Redis task queue. API spools binary to Cloudflare R2, returns HTTP 202 Accepted, and processes parsing asynchronously in dedicated worker pools.

Gap 3: In-Memory Vector Search (No Persistent Index)

- **Current State:** Chunk vectors are generated on the fly or held in memory during the RAG session.
- **Limitation:** Does not persist vector embeddings across application restarts; linear NumPy scanning cannot scale to enterprise talent pools.
- **Production Fix:** Install PostgreSQL `pgvector` extension; store chunks in a `resume_chunks` table with a `vector(768)` column and build an HNSW index using cosine distance (`vector_cosine_ops`).

Gap 4: Single-Turn RAG Without Conversational Memory

- **Current State:** The Recruiter Evidence Assistant handles independent, isolated Q&A turns.
- **Limitation:** Recruiters cannot ask follow-up questions referencing previous answers (e.g., *"Did they use Python at that company?"*).
- **Production Fix:** Introduce a Redis session dialogue buffer with an LLM query-condensation step that rewrites multi-turn questions into standalone semantic queries prior to vector retrieval.

Gap 5: Absence of Refresh Token Rotation

- **Current State:** Single JWT access token with a fixed expiration window.
- **Limitation:** When the token expires, the user experiences a hard session drop and must re-authenticate.
- **Production Fix:** Implement dual-token authentication: short-lived access tokens (15 minutes) paired with cryptographically secure, rotating refresh tokens stored in HTTP-only cookies with database revocation tracking.

Gap 6: Single-Host Docker Compose vs. Production Orchestration

- **Current State:** Docker Compose orchestrating 4 services on a single machine.
- **Limitation:** Single point of failure; no automatic horizontal pod autoscaling (HPA), automated TLS certificate renewal, or managed secrets vault.
- **Production Fix:** Author Helm charts for Kubernetes deployment, integrate AWS Secrets Manager / HashiCorp Vault, and place an edge ingress controller (Traefik/Cloudflare) with automated TLS termination in front of the cluster.

---

5. * Prioritized Interview Revision Schedules

Use these structured checklists to focus your preparation depending on how much time you have before your interview:

* 1-Week Comprehensive Revision Roadmap

```
Day 1: System Story & Business Problem
       +-- Read 01_project_story.md & README.md
       +-- Practice 60-second elevator pitch out loud 5 times
       +-- Master Hirely's dual-sided business model and problem definition

Day 2: Backend Architecture & Database
       +-- Read 03_backend_architecture.md
       +-- Review PostgreSQL models, foreign keys, and unique composite constraints
       +-- Understand SQLAlchemy 2.0 session pooling formula and Alembic migrations
       +-- Practice Q009 - Q040 in 06_interview_questions.md & 07_interview_answers_hinglish.md

Day 3: Resume Parsing & ATS Matching Formula
       +-- Read 04_ai_ml_and_rag.md (Docling & ATS Scoring sections)
       +-- Write down the 5-factor ATS formula from memory: 45/15/20/10/10
       +-- Explain why LLMs should NEVER calculate numerical match scores
       +-- Practice Q041 - Q056

Day 4: Applied AI, LLMs & Gemini Integration
       +-- Read 04_ai_ml_and_rag.md (Gemini & Prompts sections)
       +-- Review Gemini 2.5 Flash parameters, structured JSON decoding, and fallback tiers
       +-- Understand Google XYZ resume bullet rewriting prompt engineering
       +-- Practice Q057 - Q064 & Q073 - Q080

Day 5: RAG Pipeline & Substring Citation Verification
       +-- Read 04_ai_ml_and_rag.md (RAG Architecture section)
       +-- Master the 6-step RAG flow: Chunk -> Embed -> Retrieve -> Prompt -> Generate -> Verify
       +-- Explain Substring Citation Verification line by line
       +-- Practice Q065 - Q072 & Q081 - Q088

Day 6: Security, Reliability, Docker & Ownership
       +-- Read 05_security_and_reliability.md & 08_my_contribution_and_gaps.md
       +-- Review Argon2id, ZIP-bomb defense, sliding-window rate limit, and Docker health checks
       +-- Rehearse honest answer templates for frontend and technical debt
       +-- Practice Q089 - Q120

Day 7: Mock Interview Simulation
       +-- Pick 10 random questions from 06_interview_questions.md
       +-- Answer them out loud using the STAR method (Situation, Task, Action, Result)
       +-- Review Hinglish summaries in 02_project_explanation_hinglish.md for rapid concept recall
```

* 2-Day Emergency Cramming Checklist

If your interview is in 48 hours, focus **strictly** on these 6 core pillars:

```
[ ] 1. 60-Second Pitch (Memorize Word-for-Word):
    "Hirely is a high-performance recruitment intelligence platform engineered with FastAPI,
    PostgreSQL, and Google Gemini. It replaces black-box ATS keyword filters with a transparent,
    deterministic hybrid scoring engine (45% required skills, 15% preferred, 20% experience,
    10% semantic similarity, 10% completeness) and empowers recruiters with a RAG Candidate
    Evidence Assistant backed by strict substring citation verification to eliminate hallucinations.
    I owned the backend architecture, data models, AI/RAG pipelines, and Docker containerization."

[ ] 2. The 5-Component ATS Formula:
    ATS Score = 0.45*ReqSkills + 0.15*PrefSkills + 0.20*Experience + 0.10*Semantic + 0.10*Completeness
    Know WHY it's deterministic: auditability, zero token cost, zero score drift.

[ ] 3. The 6-Step RAG Flow:
    Docling Chunking (500 chars / 100 overlap) -> Gemini text-embedding-004 (768-dim) ->
    Cosine Similarity (NumPy Top-K) -> XML Bounded Prompt -> Gemini 2.5 Flash JSON Output ->
    Deterministic Substring Citation Verification against source text.

[ ] 4. Security Defense-in-Depth:
    Argon2id password hashing (memory-hard against GPUs), JWT stateless RBAC,
    magic byte inspection (%PDF-), ZIP-bomb decompression limits, sliding-window rate limiting.

[ ] 5. Top 3 Technical Limitations & Fixes:
    - Sync Docling parsing -> Fix: Celery/Redis background queue with HTTP 202 Accepted.
    - In-memory vector cosine search -> Fix: PostgreSQL pgvector with HNSW index.
    - In-memory rate limiting -> Fix: Distributed Redis sorted sets (ZADD/ZCARD).

[ ] 6. Honest Boundary Statement:
    "I owned backend engineering, database architecture, AI integration, and Docker orchestration.
    I did not build the frontend React UI; I interfaced with it via OpenAPI REST contracts."
```